

Practical Assignment 2

MOSS: Microservice-oriented Object Storage Service

1 Assignment Overview

Distributed object storage systems are fundamental components of modern cloud infrastructures, designed to manage massive amounts of unstructured data across a cluster of networked nodes. Unlike traditional file systems that organize data in a hierarchical tree structure, object storage manages data as discrete units called “objects.” Each object typically includes the data itself, a variable amount of metadata, and a unique identifier.

The primary architectural goal of these systems is to decouple the management of metadata from the actual data storage (Figure 1). This separation allows the system to scale horizontally by adding more storage nodes without creating bottlenecks at the coordination layer. Key characteristics include high availability, fault tolerance through replication, and a simplified API that allows clients to access data regardless of its physical location.

The system to be developed is a microservices-based distributed object storage platform. It utilizes gRPC for high-performance, language-agnostic communication between services. The architecture is composed of three distinct node types, each responsible for a specific layer of the storage stack.

At a high level, the distributed storage system is organized into three functional roles that separate coordination, storage, and client access.

- **Manager Node:** Acts as the central coordinator and directory. It keeps track of which storage nodes are active and where specific data is located, but it never stores the actual data files.
- **Volume Node:** Responsible for physical data persistence. It stores raw bytes on its local disk and reports its remaining capacity and health to the Manager.
- **Object Node:** Serves as the primary interface for client. It translates high-level requests (e.g., “Save this file”) into a series of steps, talking to the Manager to find a spot and then sending the data to the correct Volume.

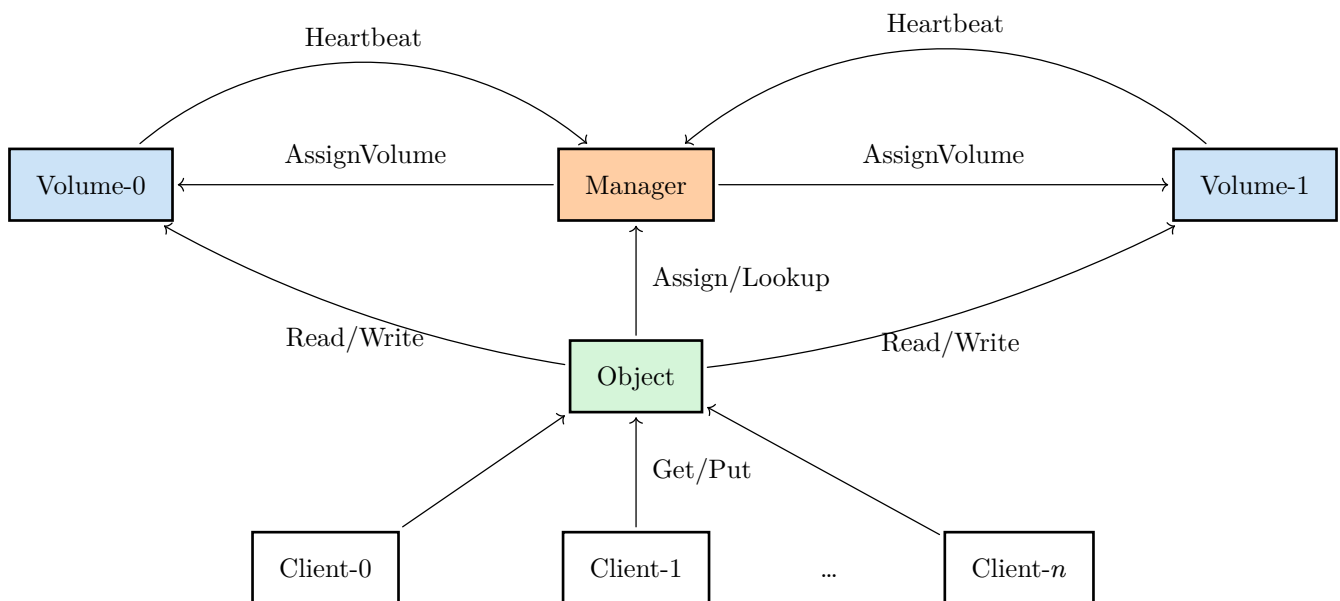


Figure 1: Overview of the MOSS system. In this configuration, there are n clients interacting with a single *Object* node. Data can be stored in two *Volume* nodes and is managed by a single *Manager*. The single *Object* node “asks” the *Manager* where to read/write the object data.

2 Objectives and Requirements

The primary objective of this practical assignment is to design and implement a distributed object storage system, using **Java** and **gRPC** as the core communication and execution models.

It is highly recommended that you carefully study the provided source code, as the classes and service definitions contain essential architectural details and patterns that are not explicitly detailed in this guide.

Each group must achieve the following objectives:

1) Model Distributed Node Interactions

Design and implement the communication lifecycles for the three core system components: Manager, Volume, and Object nodes. Students must ensure that nodes correctly handle registration, periodic heartbeats, and request orchestration through gRPC service definitions.

2) Apply Correct Synchronization Mechanisms

Use Java's concurrency primitives to ensure safe and consistent access to shared internal states, such as the volume registry in the Manager or the file lookup tables in Volume nodes. Students are expected to correctly apply mutual exclusion and thread-safe collections from the `java.util.concurrent` package.

✗ Forbidden

The use of Java's `synchronized` keyword is not allowed. You must use concurrency mechanisms available in `java.util.concurrent` (e.g., Locks, Atomic variables, Concurrent collections).

3) Reason about Distributed Correctness and Persistence

Ensure the system maintains data integrity during concurrent object uploads and retrievals. Students must implement persistent metadata management using the **MVStore** engine to handle node restarts without data loss.

4) Develop a Command-Line Interface (CLI)

Extend the provided Client node to allow users to interact with the system. The CLI must support core operations such as **Put** (uploading files), **Get** (downloading objects), and **List** (viewing stored objects), providing clear feedback on the status of distributed operations.

5) Develop a System Monitoring Dashboard (Required for groups of 3)

Implement a Graphical User Interface (GUI) that provides a comprehensive visualization of the distributed system's internal state. This dashboard must interface with the required nodes to aggregate and display:

- **Cluster Topology:** A dynamic list or visual map of all active Volume Nodes currently registered in the system, showing their network addresses and current operational health.
- **Storage Utilization Metrics:** Detailed statistics for each volume, including the remaining storage capacity and the total number of files persisted on the local disk.
- **Object Inventory:** A registry of all existing objects within the system, enabling administrators to inspect object keys and their associated metadata.

The GUI should serve as an administrative tool, abstracting the underlying gRPC communications into an intuitive monitoring experience.

3 System Intrinsics and Requirements

This section defines the core technical constraints, operational policies, and architectural boundaries for the base implementation of the distributed storage system.

3.1 Data Abstraction Layers

The system employs different abstractions for data depending on the node's role in the architecture:

- **Storage Layer (Manager & Volumes):** At this level, data is treated as an opaque **blob** identified by a unique File ID (**fid**). The Manager and Volume nodes are unaware of the original object name or user-defined metadata; they only manage raw byte sequences and their physical locations.
- **Interface Layer (Object Node):** This node manages high-level **objects**. An object consists of the data file and its associated metadata, including the logical path (or key), the total size in bytes, and the creation/upload timestamp.

3.2 Operational Constraints

To ensure predictable behavior and resource management, the following limits are enforced:

- **File Size:** Each individual file and object has a maximum size of **4 Megabytes**.
- **Volume Capacity:** Each individual volume has a maximum storage capacity of **32 Megabytes**.
- **Node Density:** A single Volume Node is permitted to manage a maximum of **5 concurrent volumes** (a total of 160 MB per node).
- **Object Bucket/Path:** An object bucket or path can be any string of length > 0 .

3.3 Management Policies

The Manager Node must implement specific strategies for resource allocation and request routing:

- **Balanced Volume Allocation:** When a new volume needs to be created, the Manager must prioritize the Volume Node with the **lowest number of currently allocated volumes**. This ensures an even distribution of storage responsibilities across the system.
- **Greedy Ticket Assignment:** For incoming **Assign** requests, the Manager should generate a ticket for the specific volume that currently reports the **highest available space** in the entire system.
- **Conservative Capacity Estimation:** When the Manager selects the best volume for a new file (the **Assign** operation), it must not use the raw capacity reported by the Volume Node. Instead, it should be conservative and assume that each volume only has **90% of its reported available space**. This provides a safety buffer to prevent volumes from becoming overfilled during concurrent write operations.
- **Node Liveness and Timeout:** The Manager must track the exact time it receives each **Heartbeat** from every Volume Node. If a Volume Node fails to send a heartbeat for **more than 20 seconds**, the Manager must temporarily ignore that node. While a node is ignored, the Manager will not assign new files to it and will not return its address during object lookups, ensuring that clients are not routed to potentially offline storage.
- **Minimal System Size:** A fully functional system requires a minimum of **two active Volume Nodes** to be operational for testing.

3.4 Architectural Boundaries

For the scope of this base implementation, several advanced distributed features are omitted to focus on core connectivity:

- **No Delete Operation:** Once an object is stored, it cannot be removed. The system operates on an append-only logic for the storage layer.
- **No Replication:** The base version of the system does not implement data redundancy. Each object is stored on exactly one Volume Node.
- **Single Coordinator (Manager):** The system supports only one active Manager Node. In a production environment, multiple managers would require a leader election protocol (e.g., Raft) which is outside the current project scope.

- **Single Interface (Object Node):** Only one Object Node is utilized in this assignment. However, because the Object Node uses a client/server database approach for metadata, it is designed to scale horizontally in future iterations.

3.5 Manager Node - `ManagerServiceProviderImpl.java`

The Manager Node must maintain a persistent and consistent state of the system, acting as the authoritative source for where data is stored and which storage nodes are healthy. On startup, the Manager must implement logic in its constructor to reload the `ledger`. Specifically, it must recover the last used File ID (`fid`) and Volume ID (`vid`) to prevent any identifier collisions in the cluster.

3.5.a Assign Function: Requesting Storage Space

The Assign operation is the first step in the object upload process. It performs the logical “placement” of a new object.

- **Requested:** The desired number of replicas for the new object. Ignore if replication is not implemented.
- **In-Between Logic:**
 - **Volume Selection:** The Manager must consult its internal volume registry to identify a set of healthy volumes that have sufficient available capacity to store the data. If there is no available capacity, it should select a Volume node and request a new volume to be created. The volume registry is stored **in-memory only**, constructed by the information from the heartbeats.
 - **ID Generation:** It must generate unique identifiers for the file (File ID) and for the new Volume IDs.
 - **Ticket Creation:** A unique “Ticket” must be constructed that will eventually be used by the client for the storage layer. The ticket has the format `<volume>:<id><cookie>` where:
 - `<volume>` - is the volume id where the file data will be stored.
 - `<id>` - unique id of the file
 - `<cookie>` - a random 32-bit number in 8 characters hex format
 - **Persistence:** The newly generated file and possible volume IDs must be stored in the persistent ledger and committed to disk to ensure they are not reused if the Manager restarts.
- **Returned:** The generated Ticket, the network address (URL) of the primary Volume Node, and the list of URLs for the secondary replication nodes.

3.5.b Lookup Function: Locating Existing Objects

The Lookup operation is used during object retrieval to resolve a file’s logical identifier into a physical network location.

- **Requested:** The files’s unique Ticket.
- **In-Between Logic:**
 - **Ticket Parsing:** The Manager must extract the Volume ID from the provided Ticket.
 - **Address Resolution:** It queries the internal node registry to find the current, live network address associated with that specific Volume ID.
- **Returned:** The network address (URL) of the Volume Node where the object is stored.

3.5.c Heartbeat Function: Volume Health Monitoring

The Heartbeat operation allows the Manager to maintain a real-time view of the system’s topology and storage health. This function is called periodically by all active Volume nodes.

- **Requested:** The Volume Node’s network address and its current storage statistics (e.g., total files, available disk space, and operational status).
- **In-Between Logic:**
 - **Registry Management:** The Manager must update the “last seen” timestamp and capacity metrics for an existing volume.

- ▶ **Health Tracking:** This information must be stored in a thread-safe data structure to allow the Assign function to make informed decisions about object placement.
- ▶ **Stale Node Detection:** Detecting nodes that have stopped sending heartbeats and removing them from the pool of active storage targets.
- **Returned:** A status code indicating successful receipt of the heartbeat.

3.6 Volume Node - `VolumeServiceProviderImpl.java`

The Volume Node handles the low-level details of the storage lifecycle, including disk I/O, local metadata persistence, and periodic health reporting to the Manager.

Upon startup, the node must scan its storage directory to reload existing volumes and their current file counts. This allows the node to recover its state and provide accurate metrics to the Manager immediately after a restart.

3.6.a AssignVolume Function: Storage Initialization

This operation prepares the node to manage a specific segment of the global storage space and establishes replication paths.

- **Requested:** A Volume ID (`vid`) and a list of URLs representing peer Volume Nodes used for data replication. The list of URLs is ignore if replication is not implemented.
- **In-Between Logic:**
 - ▶ **Maximum Volumes:** Each node can only create `MAX_NUMBER_VOLUMES` which is set to 5.
 - ▶ **Resource Allocation:** The node must create the necessary two files:
 - `<vid>.idx` - which is the list of entries stored in the volume.
 - `<vid>.data` - the actual data is stored in this file and is append only.
- **Returned:** A status code indicating successful initialization or failure (e.g., if the node has reached its `MAX_NUMBER_VOLUMES` limit).

3.6.b Write Function: Data Persistence

The Write operation is responsible for committing raw bytes to the local file.

- **Requested:** The Volume ID (`vid`), a File ID (`fid`), a security Cookie, and the raw object data in bytes.
- **In-Between Logic:**
 - ▶ **Check for duplicates:** Each file can only be written once. If it already exists, the operation fails.
 - ▶ **Add to Index:** A new entry must be appended to the `<vid>.idx`. Each entry must contain the `fid`, cookie, size and offset of the file in the `<vid>.data` file.
 - ▶ **Persistence:** The raw data must be appended to `<vid>.data`, mapped to the specific `fid` within the designated volume index. If there is not enough available space in the volume the operation fails and the Volume node must immediately report to the Manager that the volume is full.
- **Returned:** A status code confirming that the data has been safely persisted locally.

3.6.c Read Function: File Retrieval

The Read operation allows the system to fetch a previously stored file.

- **Requested:** The Volume ID (`vid`), File ID (`fid`), and the security Cookie.
- **In-Between Logic:**
 - ▶ **Identity Verification:** The node validates the Cookie to ensure the retrieval request is authorized.
 - ▶ **File Access:** Using the `vid` and `fid`, the node locates the data on the local file system and reads the content into memory.
- **Returned:** The raw data bytes and a status code indicating success or error conditions.

3.6.d Heartbeat Mechanism

The Volume Node must implement a background thread or scheduled task to maintain its “active” status in the Manager’s registry.

- **Metric Collection:** At regular intervals (e.g., 5 seconds), the node calculates its current storage usage, total file count, and available capacity against its `MAX_VOLUME_SIZE`.
- **Reporting:** It sends these metrics to the Manager via the `Heartbeat`, ensuring the Manager can include this node in future `Assign` decisions.

3.7 Object Node - `ObjectServiceProviderImpl.java`

The Object Node serves as the orchestration layer, translating high-level user operations into distributed system workflows. It maintains its own persistent index to map user-defined object keys to internal system metadata.

Upon startup, the Object node must initialize a database to maintain the mapping between user identifiers (bucket and path) and the actual storage details. This ensures that object locations are preserved across node restarts.

3.7.a Put Function: Storing New Objects

The Put operation orchestrates the end-to-end process of allocating storage, transferring data, and recording metadata.

- **Requested:** The target `bucket`, the object `path`, and the raw `data` bytes.
- **In-Between Logic:**
 - **Storage Allocation:** The Object Node first communicates with the Manager Node to “assign” a storage location, receiving a unique `Ticket` and the addresses of suitable Volume Nodes.
 - **Data Transfer:** It then streams the data to the designated primary Volume Node using the storage-layer protocol.
 - **Metadata Record:** After a successful write, the node creates an `ObjectEntry` to track the object’s state in the requested `bucket`. This entry must include the list of `Tickets` (supporting potential replication), the current `timestamp`, and the total `size` of the data.
 - **Indexing:** This `ObjectEntry` is then persisted in the local storage using a unique key derived from the `bucket` and `path`.
- **Returned:** A status code indicating the success or failure of the operation.

3.7.b Get Function: Retrieving Objects

The Get operation resolves an object’s location and retrieves its content across the system.

- **Requested:** The target `bucket` and the object `path`.
- **In-Between Logic:**
 - **Index Lookup:** The Object Node queries its local database to retrieve the `ObjectEntry` associated with the requested `bucket` and `path`.
 - **Location Resolution:** It extracts the stored `Ticket(s)` and contacts the Manager to `lookup` the current network address of the responsible Volume Node. A cache can be used to prevent unnecessary lookups when the Volume node address is already known.
 - **Data Retrieval:** The node finally retrieves the raw bytes from the identified Volume Node.
- **Returned:** The raw data bytes and the operation status.

3.7.c List Function: List Objects

The List operation gathers all objects’ metadata in a bucket.

- **Requested:** The target bucket.
- **In-Between Logic:**
 - **Index Lookup:** The Object Node queries its local database to retrieve every `ObjectEntry` associated with the bucket.
- **Returned:** The list of objects.

3.8 Client Node - `ObjectClient.java`

The Client Node serves as the primary interface for users to interact with the distributed object storage system. It provides a Command-Line Interface (CLI) that abstracts the underlying communications, allowing users to manage data through high-level operations without needing to understand the internal orchestration between the Manager and Volume nodes. The client communicates directly with an Object Node to perform storage, retrieval, and metadata queries.

Interaction with the storage cluster is performed via the `./run moss` command-line utility. By default, the client attempts to connect to an Object Node at `localhost:4281`, though the target server can be specified using the `-r` or `--remote` options.

Students must ensure that the client correctly handles system constraints, such as the maximum allowed file size for uploads (4 MiB).

3.8.a The `put` Command

The `put` command is used to upload a local file into the distributed system.

- **Syntax:** `./run moss put [-b <bucket>] <local_file> <remote_path>`
- **Parameters:**
 - `-b <bucket>`: Specifies the target logical bucket (defaults to “sd”).
 - `<local_file>`: The path to the source file on the user’s local file system.
 - `<remote_path>`: The unique path or name assigned to the object within the storage system.

3.8.b The `get` Command

The `get` command retrieves an existing object from the system and saves it to the local disk.

- **Syntax:** `./run moss get [-b <bucket>] <remote_path> <local_file>`
- **Parameters:**
 - `-b <bucket>`: The bucket where the object is currently stored.
 - `<remote_path>`: The name/path of the object to be fetched from the cluster.
 - `<local_file>`: The destination path where the retrieved content will be saved locally.

3.8.c The `list` Command

As part of the assignment, students must implement a `list` functionality. This command should allow users to browse the contents of a bucket, providing an inventory of all stored objects and their associated metadata.

3.9 Graphical User Interface (GUI)

The development of a Graphical User Interface (GUI) is a core requirement for groups of three students. This application serves as an administrative dashboard, providing a real-time visualization of the cluster’s internal state and storage distribution. The GUI must consolidate information from both the Manager and Object nodes to present a unified view of the system.

GUI Functional Requirements

The dashboard should provide intuitive visualizations for the following system aspects:

- **Cluster Health and Topology:** A visual representation of all registered Volume Nodes, indicating their online/offline status based on recent heartbeat activity.

- **Storage Distribution:** A detailed breakdown of each volume's utilization, showing the percentage of the 32 MiB capacity currently in use and the number of stored blobs.
- **Object Inventory:** A searchable list of all objects managed by the Object Node, displaying their logical keys, sizes, and timestamps.

Technical Requirement: The Manager **state** Function

To support the monitoring dashboard, students must extend the Manager Node by implementing a new endpoint called **state**. This function is responsible for exposing the cluster's current configuration and health metrics.

The GUI application will periodically invoke this **state** function to update its visualizations, ensuring that the administrator has an accurate view of the distributed system's performance and capacity.

3.10 Extra Tasks

Students are encouraged to implement additional features to further explore concurrency and distributed systems challenges. These enhancements contribute to the "Innovation" score. For this assignment two extra functionalities can be implemented:

- **Object Deletion::** Implement the capability to delete an object and respective file from the assigned volume. The following must be implemented:
 - **Delete:** Add a delete function to the Volume node.
 - **Remove:** Add a remove function to the Object node. A remove function will trigger a delete function in the proper Volume Node.
 - **File tombstones:** Because the volume index is an append only list, you must mark a file for deletion with a tombstone.
 - **Compact:** Add a compact function to the Volume node. This function will recreate the volume data but without the deleted file(s). When compacting, the volume must be put offline until it finishes.
- **Replication:** Implement the capability to replicate files with consistency. The following must be implemented:
 - **Replication on request:** The Object node can be configured to store the object with replication.
 - **Volume Replication:** Data can only be replicated on volumes that are also replicated. For example, a two replica policy, requires that a volume with ID **vid** must exist in two different Volume nodes.
 - **Replication Responsibility:** It is the responsibility of the main Volume to replicate the data. The Manager, on assign, will reply with a main Volume and a list of replica volumes.

4 Architectural Constraints and Provided Base Code

All groups **must adhere to the proposed reference architecture** provided with this assignment. To ensure consistency, comparability of results, and a common baseline for evaluation, a **base code skeleton** implementing this architecture will be supplied to each group at the start of the project.

The provided source code establishes a skeletal framework for a microservices-based distributed object storage system. It utilizes a layered, architecture that separates network communication, domain logic, and persistence.

The template includes the following architectural components:

- **gRPC Service Definitions:** Pre-defined Protocol Buffer contracts for the Manager, Volume, and Object services, establishing the communication protocols for the entire cluster.
- **Service Adapters and Implementation Skeletons:** Java boilerplate for gRPC server implementations (Services) and client stubs (Adapters), mapping network messages to domain models.
- **Automated Service Discovery:** A discovery mechanism (`GrpcServiceDiscovery`) that simplifies the process of nodes finding and connecting to one another within the local network.
- **Command-Line Tooling:** A functional `ObjectClient` using the `picocli` library, enabling immediate testing of the `put` and `get` workflows from the terminal.

- **Network and Runtime Utilities:** Helper classes for retrieving local IP addresses and executing gRPC stubs with standard error handling.

Groups are **allowed to extend, or optimize** the proposed architecture, including:

- Introducing new classes or interfaces.
- Reorganizing internal components.
- Replacing internal synchronization strategies.

However, the following constraints are **mandatory**:

- **Execution compatibility:** All defined execution options must be preserved exactly as specified in the base code. The nodes must be runnable using the same commands and parameters as the reference implementation.
- **Output compatibility:** Any required output format, semantics, and content defined in the base code must be maintained. This ensures that automated testing and evaluation tools can be applied uniformly across all submissions.
- **Architectural compliance:** The overall architectural roles (manager, volume, object, cli, GUI) must remain identifiable and consistent with the reference design, even if their internal implementation changes.
- **Behavioral equivalence:** Any architectural modification must preserve the observable behavior of the system with respect to the system constraints.

Failure to comply with the provided architecture, execution options, or required outputs may result in the solution being **non-evaluable** or **penalized**, regardless of the internal quality of the implementation.

5 Evaluation

The evaluation of this project is divided into two main components: the technical implementation (90%) and a final presentation (10%). **The presentation is mandatory.** All groups must maintain their source code in a dedicated GitHub repository, which must include a `README.md` file documenting the architectural decisions and concurrency strategies employed.

5.1 Presentation (10%)

Following the code submission, each group will perform a presentation. The total duration is strictly limited to **15 minutes**, structured as follows:

- **Presentation (5 min):** A concise overview of the implemented system.
- **Demo & Q&A (10 min):** A live demonstration of the software, during which the instructor may ask technical questions regarding the implementation.

5.2 Technical Implementation (90%)

The submitted project will be evaluated based on the following weighted criteria, focusing on technical correctness, software engineering standards, and documentation quality.

Evaluation Criteria

- **Functionality (55%)**
 - All core features are working correctly (45%)
 - No data inconsistency is detected (10%)
- **Code Quality (15%)**
 - Proper use of object-oriented programming and modularity (10%)
 - Code readability and comments (5%)
- **Documentation (15%)**
 - Complete and clear README (10%)
 - Instructions on how to compile and run the application.

- An explanation of the implementation of the different nodes.
- How the group specifically addressed concurrency.
- Other information considered relevant by the group.
- Technical documentation quality (5%)
- **Innovation (15%)**
 - Implementation of optional features
 - Object Deletion (5%)
 - Object Replication (10%)
 - Other (5%)
 - Creative solutions to problems
 - User experience enhancements
- **No GUI Implementation (−15%)**
 - **Only for groups of three students**
 - Failure to implement a GUI will result in a 15% penalty.
 - A well-designed and fully implemented GUI incurs no penalty.

5.3 Academic Integrity Guidelines

This assignment is a **collaborative group effort**. While you are encouraged to discuss general concepts with peers from other teams, your final codebase must be the **unique, original creation** of your specific group.

Usage of AI & External Tools

- **Permitted:** Using AI assistants to troubleshoot bugs or clarify complex theoretical concepts.
- **Prohibited:** Using AI to generate entire solutions or major functional blocks of code.
- **Citations:** You must explicitly credit any external code snippets or concepts sourced from the web or elsewhere.

Standards & Consequences

- **Formatting:** All references and citations must strictly adhere to **IEEE templates**.
- **Plagiarism:** There is a zero-tolerance policy for academic dishonesty.
- **Penalty:** Any group found in violation of these rules will face **immediate disqualification** of their work.

6 Delivery

To ensure a smooth evaluation process, students must adhere to the following delivery requirements. The state of the repository at the deadline will be considered the final submission for grading.

6.1 Submission Requirements

The project delivery consists of two mandatory components hosted on GitHub:

- **Source Code:** A fully functional Java project.
- **Documentation:** A `README.md` file located in the root directory.

6.2 Deadline and Protocol

The submission window closes on **May 31 at 23:59**.

- **Automated Retrieval:** At the deadline, the teaching staff will pull the code directly from the main branch of your GitHub repository.
- **Late Submissions:** Any delays must be **explicitly declared** by the group via email before the deadline.
- **Version Control:** We strongly recommend frequent commits. The commit history will be used to verify the group's contribution distribution and the evolution of the solution.

Warning

Code pushed after the deadline without a prior, approved declaration will not be considered for evaluation.